

Practical 1 — Tokenization, Stemming, Lemmatization (NLTK)

Objective: Perform different tokenization techniques, apply stemming (Porter, Snowball), and implement lemmatization (WordNet) with optional POS-aware lemmatization.

Sample Sentence:

"NLTK is a leading platform for building Python programs to work with human language data! Let's explore tokenization, stemming, and lemmatization :) #NLP"

```
In [6]: # Sample sentence
sentence = "NLTK is a leading platform for building Python programs to work with hu
print('Sentence:', sentence)
```

Sentence: NLTK is a leading platform for building Python programs to work with human language data! Let's explore tokenization, stemming, and lemmatization :) #NLP

```
In [ ]: # Imports and NLTK downloads (run once)
import nltk
# The following downloads small corpora/models used by tokenizers/lemmatizers.
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('omw-1.4')
from nltk.tokenize import TweetTokenizer, WordPunctTokenizer, TreebankWordTokenizer
from nltk.tokenize import MWETokenizer
from nltk.stem import PorterStemmer, SnowballStemmer
from nltk.stem import WordNetLemmatizer
from nltk import pos_tag
from nltk.corpus import wordnet

try: # GPU device detection (use CUDA if available)
    import torch
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print('device =', device)
except Exception:
    device = 'cpu'
    print('torch not installed; using CPU')

def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
```

```
else:
    return wordnet.NOUN
```

```
[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\jenil\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]   C:\Users\jenil\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   C:\Users\jenil\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]   C:\Users\jenil\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
device = cuda
```

```
In [ ]: # 1) Tokenization examples
# Whitespace tokenization
ws_tokens = sentence.split()
# Punctuation-based tokenization (WordPunct)
wp = WordPunctTokenizer()
wp_tokens = wp.tokenize(sentence)
# Treebank (handles contractions)
tb = TreebankWordTokenizer()
tb_tokens = tb.tokenize(sentence)
# Alternatively: word_tokenize uses the Treebank tokenizer under the hood
treebank_tokens = word_tokenize(sentence)
# Tweet tokenizer (handles hashtags, emoticons)
tt = TweetTokenizer()
tweet_tokens = tt.tokenize(sentence)
# MWE tokenization: group ('human', 'language') into a single token
mwe = MWETokenizer([('human', 'language')])
mwe_input = word_tokenize(sentence)
mwe_tokens = mwe.tokenize(mwe_input)
# Display results
print('Whitespace tokens:', ws_tokens)
print('WordPunct tokens:', wp_tokens)
print('Treebank tokens (TreebankWordTokenizer):', tb_tokens)
print('Treebank tokens (word_tokenize):', treebank_tokens)
print('Tweet tokens:', tweet_tokens)
print('MWE tokens (human_language grouped):', mwe_tokens)
```

```

Whitespace tokens: ['NLTK', 'is', 'a', 'leading', 'platform', 'for', 'building', 'Pytho
n', 'programs', 'to', 'work', 'with', 'human', 'language', 'data!', "Let's", 'exp
lore', 'tokenization,', 'stemming,', 'and', 'lemmatization', ':)', '#NLP']
WordPunct tokens: ['NLTK', 'is', 'a', 'leading', 'platform', 'for', 'building', 'Pyt
hon', 'programs', 'to', 'work', 'with', 'human', 'language', 'data', '!', 'Let',
'", 's', 'explore', 'tokenization', ',', 'stemming', ',', 'and', 'lemmatization',
':)', '#', 'NLP']
Treebank tokens (TreebankWordTokenizer): ['NLTK', 'is', 'a', 'leading', 'platform',
'for', 'building', 'Python', 'programs', 'to', 'work', 'with', 'human', 'language',
'data', '!', 'Let', "'s", 'explore', 'tokenization', ',', 'stemming', ',', 'and', 'l
emmatization', ':', ')', '#', 'NLP']
Treebank tokens (word_tokenize): ['NLTK', 'is', 'a', 'leading', 'platform', 'for',
'building', 'Python', 'programs', 'to', 'work', 'with', 'human', 'language', 'data',
'!', 'Let', "'s", 'explore', 'tokenization', ',', 'stemming', ',', 'and', 'lemma
tization', ':', ')', '#', 'NLP']
Tweet tokens: ['NLTK', 'is', 'a', 'leading', 'platform', 'for', 'building', 'Pytho
n', 'programs', 'to', 'work', 'with', 'human', 'language', 'data', '!', "Let's", 'ex
plore', 'tokenization', ',', 'stemming', ',', 'and', 'lemmatization', ':)', '#NLP']
MWE tokens (human_language grouped): ['NLTK', 'is', 'a', 'leading', 'platform', 'fo
r', 'building', 'Python', 'programs', 'to', 'work', 'with', 'human_language', 'dat
a', '!', 'Let', "'s", 'explore', 'tokenization', ',', 'stemming', ',', 'and', 'lemma
tization', ':', ')', '#', 'NLP']

```

```

In [ ]: # 2) Stemming (Porter vs Snowball)
porter = PorterStemmer()
snowball = SnowballStemmer('english')
# Prepare a list of candidate words (filter out pure punctuation)
candidates = [t for t in treebank_tokens if any(c.isalnum() for c in t)]
print('Candidates for stemming:', candidates)
porter_results = [(w, porter.stem(w)) for w in candidates]
snowball_results = [(w, snowball.stem(w)) for w in candidates]
print('Porter Stemmer results:')
for orig, stem in porter_results:
    print(f'{orig} -> {stem}')
print('Snowball Stemmer (English) results:')
for orig, stem in snowball_results:
    print(f'{orig} -> {stem}')

```

```
Candidates for stemming: ['NLTK', 'is', 'a', 'leading', 'platform', 'for', 'building', 'Python', 'programs', 'to', 'work', 'with', 'human', 'language', 'data', 'Let', "'s", 'explore', 'tokenization', 'stemming', 'and', 'lemmatization', 'NLP']
```

Porter Stemmer results:

```
NLTK -> nltk
is -> is
a -> a
leading -> lead
platform -> platform
for -> for
building -> build
Python -> python
programs -> program
to -> to
work -> work
with -> with
human -> human
language -> languag
data -> data
Let -> let
's -> 's
explore -> explor
tokenization -> token
stemming -> stem
and -> and
lemmatization -> lemmat
NLP -> nlp
```

Snowball Stemmer (English) results:

```
NLTK -> nltk
is -> is
a -> a
leading -> lead
platform -> platform
for -> for
building -> build
Python -> python
programs -> program
to -> to
work -> work
with -> with
human -> human
language -> languag
data -> data
Let -> let
's -> 's
explore -> explor
tokenization -> token
stemming -> stem
and -> and
lemmatization -> lemmat
NLP -> nlp
```

```
In [ ]: # 3) Lemmatization (WordNet)
        lemmatizer = WordNetLemmatizer()
        # Lemmatize without POS (default assumes nouns)
        lemmatized_default = [(w, lemmatizer.lemmatize(w)) for w in candidates]
```

```
# POS tagging + POS-aware lemmatization for better results
pos_tags = pos_tag(candidates)
lemmatized_pos = []
for word, tag in pos_tags:
    wn_tag = get_wordnet_pos(tag)
    lem = lemmatizer.lemmatize(word, pos=wn_tag)
    lemmatized_pos.append((word, tag, wn_tag, lem))

print('Lemmatization (default noun):')
for orig, lem in lemmatized_default:
    print(f'{orig} -> {lem}')
print('POS-tagged lemmatization results:')
for orig, tag, wn_tag, lem in lemmatized_pos:
    print(f'{orig} ({tag}) -> {lem} [WN POS: {wn_tag}]')
```

```

Lemmatization (default noun):
NLTK -> NLTK
is -> is
a -> a
leading -> leading
platform -> platform
for -> for
building -> building
Python -> Python
programs -> program
to -> to
work -> work
with -> with
human -> human
language -> language
data -> data
Let -> Let
's -> 's
explore -> explore
tokenization -> tokenization
stemming -> stemming
and -> and
lemmatization -> lemmatization
NLP -> NLP
POS-tagged lemmatization results:
NLTK (NNP) -> NLTK [WN POS: n]
is (VBZ) -> be [WN POS: v]
a (DT) -> a [WN POS: n]
leading (VBG) -> lead [WN POS: v]
platform (NN) -> platform [WN POS: n]
for (IN) -> for [WN POS: n]
building (VBG) -> build [WN POS: v]
Python (NNP) -> Python [WN POS: n]
programs (NNS) -> program [WN POS: n]
to (TO) -> to [WN POS: n]
work (VB) -> work [WN POS: v]
with (IN) -> with [WN POS: n]
human (JJ) -> human [WN POS: a]
language (NN) -> language [WN POS: n]
data (NNS) -> data [WN POS: n]
Let (NNP) -> Let [WN POS: n]
's (POS) -> 's [WN POS: n]
explore (VB) -> explore [WN POS: v]
tokenization (NN) -> tokenization [WN POS: n]
stemming (VBG) -> stem [WN POS: v]
and (CC) -> and [WN POS: n]
lemmatization (NN) -> lemmatization [WN POS: n]
NLP (NNP) -> NLP [WN POS: n]

```

Conclusion

- Different tokenizers produce different token sets; Treebank and Tweet tokenizers handle contractions and social-media tokens better than naive whitespace splitting.

- Stemmers (Porter, Snowball) reduce words to roots but sometimes produce non-words; Snowball often gives slightly cleaner stems for English.
- Lemmatization (WordNet) returns dictionary base forms; POS-aware lemmatization improves accuracy for verbs/adjectives/adverbs.

This notebook demonstrates practical preprocessing steps important for downstream NLP tasks.